

Decentralized Lattice-Based Device-to-Device Authentication for the Edge-Enabled IoT

Ali Shahidinejad  and Jemal Abawajy , *Senior Member, IEEE*

Abstract—In this article, we present a novel and decentralized authentication and key exchange protocol for device-to-device (D2D) communications in the Internet of Things (IoT). The existing lattice-based D2D authentication protocols suffer from efficiency, security and privacy including high storage cost, lack of strong anonymity, and high key revocation overheads. The proposed protocol is designed to overcome these challenges by using lattice-based cryptography, edge computing, and blockchain technology. Edge computing is used to reduce computation overhead on IoT devices, and blockchain is used to maintain public keys to simplify key revocation and interdomain authentication. We have analyzed thoroughly the security of the proposed protocol using both the informal and formal security methods. The proposed protocol is resilient to several known classical attacks, such as the ephemeral secret leakage attack and public-key quantum attack. We have also conducted a thorough performance analysis and compared it to several existing approaches. The results show that the proposed protocol decreases the device-side computation overhead by approximately 58% as compared to other approaches. In addition, the proposed protocol shows approximately 2% decrease in the communication overhead compared to just post-quantum protocols.

Index Terms—Authentication, blockchain, edge computing, Internet of Things (IoT), lattice.

I. INTRODUCTION

THE Internet of Things (IoT) refers to a collection of networked physical devices that are embedded with sensing, computing power, and networking capabilities. The IoT devices have limited resources, such as limited storage capacity, which restrict the device's ability to store, analyze, and process data. To ameliorate these limitations, IoT devices use edge computing [1] to process and analyze data locally as well as to minimize delays and accelerate decision making [2].

Direct device communication is a crucial element of the IoT. Diverse use cases for device-to-device (D2D) communication in the IoT include smart homes, industrial automation, health care, and vehicle-to-vehicle communication. D2D communication allows real-time communication and data sharing without relying on centralized servers [3]. This enhances efficiency, reduces latency, and improves reliability, especially in areas where network connectivity is restricted or unstable. However,

these benefits pose security and privacy risks because there is no entity to verify the authenticity of the IoT devices. Furthermore, in the absence of appropriate secure communication methods, sensitive data are subject to misuse. As a result, a reliable authentication and key agreement are critical for verifying device IDs and generating shared secret keys for data encryption [4].

Several kinds of centralized and decentralized authentication and key agreement protocols have been proposed for securing D2D communications [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. Centralized authentication protocols rely on a single central authority, which is vulnerable to a single point of failure and an easy target for a range of security and privacy attacks. In contrast, decentralized authentication protocols distribute trust among all the devices in the network and are suitable for the IoT environment. However, the existing protocols suffer from high communication overhead and are fallible to several known classical attacks, such as the ephemeral secret leakage attack, key escrow issues, and quantum attack [15].

One promising option for achieving quantum-proof security is lattice-based authentication, which relies on mathematical problems that are considered difficult for quantum computers to solve. Although lattice-based D2D authentication protocols offer strong security guarantees [16], they present significant challenges for resource-constrained IoT devices. The existing lattice-based solutions do not provide unlinkability property for IoT devices. In addition, these solutions require each IoT device to keep the public key of all the IoT devices in the network. This requirement leads to significant storage costs because the size of the public keys used in lattice-based cryptography is large [16], [17]. In addition, key revocation can be prohibitively costly, as every IoT device must communicate with its edge or cloud server to update public keys. Furthermore, the existing D2D authentication protocols do not support both the intradomain and interdomain authentication securely. Therefore, there is still a need for a lightweight and efficient D2D authentication protocol that offers long-term strong security guarantees and is suitable for resource-constrained IoT devices.

The aforementioned challenges have motivated us to develop an efficient and secure D2D authentication and key exchange protocol. The proposed protocol is resilient to several known classical security attacks like ephemeral secret leakage and post-quantum security attacks, such as public-key quantum attacks. Furthermore, its computation overhead is significantly lower than the baseline classical and post-quantum protocols. Fig. 2 visually highlights the proposed protocol, and its key contributions are summarized as follows.

Manuscript received 14 March 2023; revised 20 July 2023 and 4 September 2023; accepted 15 September 2023. Date of publication 12 October 2023; date of current version 12 December 2023. (Corresponding author: Ali Shahidinejad.)

The authors are with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia (e-mail: a.shahidinejad@research.deakin.edu.au; jemal.abawajy@deakin.edu.au).

Digital Object Identifier 10.1109/JSYST.2023.3319280

TABLE I
COMPARING THE RELATED STUDIES

Ref.	Tech.	App.	Architecture	Advantages	Drawbacks
Post-quantum Protocols					
[6]	Lattice	IoT	Centralized	Low communication cost.	High storage and computation costs, Not anonymous and unlinkable, Single domain.
[18]	Lattice	IoT	Centralized	Quantum proof.	High storage cost, Key escrow, Not anonymous and unlinkable, Single domain.
[8]	Lattice	IoV	Centralized	Anonymous, Quantum proof.	High storage cost, Key escrow, Single domain.
Blockchain-based Protocols					
[19]	ECC	IoV	Decentralized	Anonymous, Cross domain.	High communication overhead, Not quantum-proof, Key escrow, Vulnerable to ephemeral secret leakage attack.
[5]	ECC	IIoT	Decentralized	Anonymous, Unlinkable, Cross domain.	Vulnerable to ephemeral secret leakage and DDoS attacks, High communication overhead, Not quantum-proof.
Classical Protocols					
[9]	ECC	IoD	Centralized	Anonymous.	Vulnerable to DDoS, Single domain, Not quantum-proof.
[10]	ECC	IoD	Centralized	Low communication cost	Single domain, Not quantum-proof, Not anonymous and unlinkable.
[11]	ECC	IoD	Centralized	Low communication cost, Anonymous.	Key escrow, Single domain, Not quantum-proof.
[12]	ECC	IoV	Centralized	Anonymous, Unlinkable.	Not quantum-proof, Single domain, Difficult public key revocation.
[20]	PUF	IoD	Centralized	Anonymous, Unlinkable, Low computation overhead.	No perfect forward secrecy, Single domain, Vulnerable to ephemeral secret leakage attack.
[7]	PUF	IoD	Centralized	Low computation overhead, Anonymous, Unlinkable.	No perfect forward secrecy, Single domain, Vulnerable to ephemeral secret leakage attack.
[21]	Bilinear Pairing	IoT	Centralized	Anonymous.	High computation cost, Not quantum-proof, Single domain, Key escrow.
[22]	Bilinear Pairing	IoV	Centralized	Anonymous.	High computation cost, Not quantum-proof, Single domain.
Post-quantum and Blockchain-based Protocol					
Our	Lattice	IoT	Decentralized	Anonymous, Unlinkable, Cross domain, Low storage overhead.	Higher communication cost than classic approaches.

- 1) A blockchain-based framework for D2D communication in an edge-enabled IoT environment that reduces the computation and storage overheads on IoT devices.
- 2) A decentralized-lattice-based IoT D2D authentication protocol that can support secure interdomain communications, unlinkability of IoT devices, and key revocation.
- 3) A rigorous verification of the authentication protocol's security using both the informal and formal security approaches.
- 4) An implementation of the D2D authentication protocol on the IoT test bed using the "ESP32-DevKitC" board.
- 5) A comparative analysis of the proposed protocol against classical, blockchain-based, and post-quantum methods.

The rest of this article is organized as follows. We discuss related work in Section II, followed by system and adversary models in Section III. The proposed authentication protocol is described in Section IV. The security and performance analyses of the proposed protocol are presented in Sections V and VI, respectively. Finally, Section VII concludes this article.

II. RELATED WORK

In recent years, various approaches have been proposed for D2D authentication in different application domains, such as IoT, Industrial Internet of Things (IIoT), and Internet of Vehicles (IoV). Table I summarizes the comparison of related studies

based on the techniques used, application domains, architectures, advantages, and drawbacks.

Shen et al. [19] proposed a blockchain-based authentication approach using elliptic curve cryptography (ECC) in the IoV domain. The proposed approach is decentralized, anonymous, and cross domain, but it suffers from high communication overhead, vulnerability to ephemeral secret leakage attacks, and key escrow issues. Zhang et al. [5] also proposed a decentralized approach based on ECC for D2D authentication in the IIoT domain. Their approach provides anonymity and unlinkability across domains, but it is vulnerable to distributed denial-of-service (DDoS) attacks and suffers from high communication overhead.

Semal et al. [9] proposed a certificateless ECC-based approach for device authentication in the Internet of Drones (IoD) domain. Their approach is centralized and provides anonymity but is vulnerable to DDoS attacks and is not quantum proof. Bera et al. [10] proposed a low-communication-cost ECC-based approach for D2D authentication in the IoD domain, but their approach is not anonymous, unlinkable, and not quantum proof. Irshad et al. [11] proposed an anonymous and low-communication-cost ECC-based approach for D2D authentication in the IoD domain, but their approach suffers from key escrow issues and is not quantum proof. Eftekhari et al. [12] proposed an anonymous and unlinkable ECC-based approach for D2D authentication in the IoV domain, but their approach

is not quantum proof and suffers from difficulties in public key revocation.

Pu et al. [7] and Bansal and Sikdar [20] proposed D2D authentication approaches using physical unclonable functions (PUFs) in the IoD domain. Their approaches provide anonymity, unlinkability, and low computation overhead but suffer from the lack of perfect forward secrecy and vulnerability to ephemeral secret leakage attack.

Gupta and Biswas [6] proposed a lattice-based approach for D2D authentication in the IoT domain, which has low communication cost, but suffers from high storage and computation costs and is not anonymous and unlinkable. Islam and Zeadally [18] proposed a quantum-proof lattice-based approach for device authentication for the IoT, but their approach suffers from key escrow problem and is not anonymous and unlinkable. Gupta et al. [8] proposed a lattice-based approach for D2D authentication for the IoV, which is anonymous and quantum proof, but suffers from high storage cost and key escrow problem.

Naresh et al. [21] proposed a bilinear-pairing-based approach for D2D authentication in the IoT domain, which provides anonymity but suffers from high computation cost, lack of quantum-proof, single domain, and key escrow problem. Elkhailil et al. [22] proposed an anonymous bilinear-pairing-based solution. Nevertheless, these approaches place a significant computational burden on the devices.

From a broad perspective, classical cryptographic methods employed for securing D2D communication, such as [9], [10], [11], [12], [13], and [14], which rely on public key encryption (e.g., ECC), are susceptible to quantum attacks. Therefore, it is crucial to adopt a quantum-proof authentication protocol to ensure long-term security for IoT devices. Furthermore, these methods are centralized and possess a single point of failure issue. Blockchain-based solutions, such as [19] and [5], have resolved the single point of failure issue. Nevertheless, they impose substantial overheads on devices, lack strong anonymity, and are susceptible to quantum attacks. Finally, current lattice-based solutions, such as [6], [8], [18] and [23], incur high storage overheads on IoT devices, lack support for interdomain authentication, and make key revocation difficult. In conclusion, the existing classical and post-quantum protocols still have some important limitations that motivated us to propose a new blockchain-based protocol. Our research stands out from prior studies by utilizing blockchain technology to tackle the single point of failure issue, mitigate storage overheads on devices, and simplify the key revocation process. Furthermore, our approach involves the delegation of resource-intensive computation tasks to edge servers while simultaneously providing strong anonymity. This enables IoT devices to operate with minimal resource consumption while maintaining a high level of security and privacy.

III. SYSTEM AND ADVERSARY MODELS

This section describes the proposed blockchain-based IoT network system and the adversary model used to evaluate the authentication protocol's security.

A. System Model

The system model presented in this research is a blockchain-based edge-enabled IoT network. Unlike traditional centralized systems, there is no central authority, such as a cloud to manage the devices. Instead, the system model consists of a decentralized network of edge servers that store the public keys of IoT devices, similar to [24], [25], and [26]. IoT devices are physical devices that collect data and send it to the edge servers. Each IoT device has a unique public key that is used during the authentication procedure for signing messages. The edge servers serve as the blockchain network's initial point of interaction, responsible for housing and managing the public keys associated with the IoT devices to which they are connected. The blockchain network maintains a ledger of all the public keys of IoT devices, ensuring the integrity and security of the data. This ledger enables the network to have a decentralized public record of all the IoT devices connected to the network. Any edge server connected to the network can access this ledger, eliminating the need for a centralized authority to manage the devices. To maintain the accuracy of the ledger, a consensus protocol is used by the blockchain network to verify transactions. This protocol ensures that all the nodes on the network agree on the state of the ledger. In addition, the blockchain network uses smart contracts to automate the retrieval of public keys for verifying the authenticity of IoT devices. These smart contracts also facilitate the key revocation process, making it possible to remove devices from the network if necessary.

The proposed system model can be effectively implemented in various domains, such as IoV, IIoT, and health care, where fog/edge servers are utilized. It is important to note that D2D communication is direct communication between two devices, and only the processes of mutual authentication and key agreement are performed through the edge servers. This is due to the limitations of resource-constrained IoT devices, which may not be able to interact with the blockchain network as they require an operating system. Besides, by relying on edge servers for public key retrieval and signature verification, we can significantly reduce the computation overhead on IoT devices. Fig. 1 illustrates the system architecture of the proposed protocol.

B. Adversary Model

We use the Canetti–Krawczyk (CK) adversary model [27] to evaluate the security of the proposed authentication protocol. We chose the CK model because it is one of the most common and robust security models that have been widely used in comparable protocols and are largely accepted by researchers in this field. This model gives an adversary the ability to intercept, modify, and inject messages in the communication between the legitimate parties. In addition, the adversary may obtain short- or long-term keys. The goal of the adversary is to impersonate a legitimate party or to disrupt the communication between the legitimate parties. The CK model is used to determine the level of security that a protocol must provide to withstand the capabilities of an adversary. Note that the protocol needs to provide resiliency against the CK adversary when we have quantum computers. Table II outlines the specific actions that

TABLE II
DESCRIPTION OF ADVERSARY CAPABILITIES

Query	Description
$EXE()$	By sending this query, $\mathcal{A}$ gets the communicating messages.
$SND()$	By sending this query, $\mathcal{A}$ gets a response according to his/her submitted message.
$HSH()$	By sending this query, $\mathcal{A}$ gets the hashed value of the input.
$ID()$	By sending this query, $\mathcal{A}$ gets the real identifier of the input.
$LTSR()$	By sending this query, $\mathcal{A}$ gets the long-term secrets of the input.
$STSR()$	By sending this query, $\mathcal{A}$ gets the short-term secrets of the input.
$SKR()$	By sending this query, $\mathcal{A}$ gets the session key.
$TST()$	By sending this query, $\mathcal{A}$ gets the session key or a same-sized random value randomly.

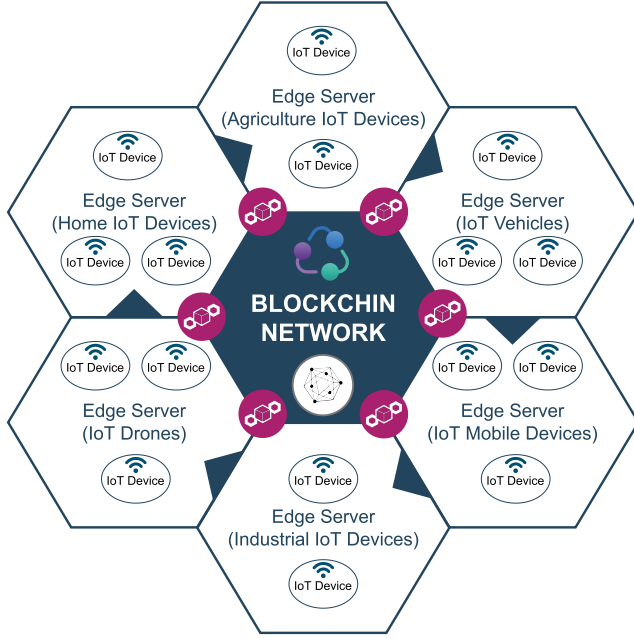


Fig. 1. System architecture of the proposed protocol.

an adversary $\mathcal{A}$ can take, represented in the form of queries. These capabilities may include gaining access to communication messages, sending arbitrary messages, and obtaining either short- or long-term keys.

IV. DESIGN OF THE PROPOSED PROTOCOL

In this section, we discuss the proposed authentication protocol. We begin by presenting a brief overview of the fundamental concepts and mathematical definitions used in the proposed authentication protocol design.

Lattices have proven effective in constructing cryptographic primitives resistant to quantum attacks. An integer lattice $\mathcal{L}$ can be defined as a periodic grid in $\mathbb{Z}^m$, encompassing integer linear combinations of vectors derived from a basis $\mathbf{B} = \mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$. The bilateral inhomogeneous small integer solution (Bi-ISIS) and the computational bilateral inhomogeneous small integer solution (Cbi-ISIS) are specific problems within lattice-based cryptography that involve finding solutions to a system of linear equations with small integer coefficients. These problems are considered inhomogeneous problems because the

system of equations includes a constant term or a nonzero vector on the right-hand side.

The proposed protocol is developed within the context of the Bi-ISIS and Cbi-ISIS problems. Numerous prior research works have leveraged these problems to develop authentication schemes that are resistant to quantum attacks [8], [18], [28], [29], [30], [31]. Bi-ISIS and Cbi-ISIS are formally defined as follows.

Definition 1 (Bi-ISIS): Assuming a square q -ray matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times n}$, $\beta \in \mathbb{Z}^+$, and $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{Z}_q^n$, it is computationally hard to find $\mathbf{w}, \mathbf{y} \in \mathbb{Z}^n$ that satisfies $\mathbf{M}\mathbf{w} = \mathbf{x}_1 \bmod q$, where $\|\mathbf{w}\| \leq \beta$ and $\mathbf{y}^T \mathbf{M} = \mathbf{x}_2^T \bmod q$, where $\|\mathbf{y}\| \leq \beta$ [17].

Definition 2 (Cbi-ISIS): Assuming a square q -ray matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times n}$, $\beta \in \mathbb{Z}^+$, and $\mathbf{x}_1 = \mathbf{M}\mathbf{w}, \mathbf{x}_2 = \mathbf{y}^T \mathbf{M} \in \mathbb{Z}_q^n$, where $\|\mathbf{w}\|, \|\mathbf{y}\| \leq \beta$, it is computationally hard to obtain $\mathbf{y}^T \mathbf{M}\mathbf{w} \bmod q \in \mathbb{Z}^n$ [17].

Fig. 2 illustrates the overview of the proposed protocol. The protocol consists of three distinct phases: blockchain design, IoT device registration, and authentication/key agreement. In the blockchain design phase, edge servers create a private blockchain network using Hyperledger Fabric, with the distributed ledger of the network serving as secure storage for public keys. During the registration phase, each IoT device is registered by its edge server, and its public keys are added to the blockchain ledger. In the authentication and key agreement phase, registered IoT devices can authenticate each other and generate a shared session key. The proposed protocol leverages the distributed ledger for public key retrieval, ensuring that the edge servers can verify the authenticity and validity of public keys. This article provides detailed information on the specifics of each phase. In the following subsections, each phase is described in detail.

A. Blockchain Design

During this stage, a private blockchain platform known as Hyperledger Fabric [32] is used to form a consortium blockchain network among edge servers. In this network, each edge server acts as an organization that owns a copy of the distributed ledger on its peer nodes. These servers can transmit transactions, which can involve writing, updating, or revoking a public key, to the network. The use of consortium-based blockchain is examined in this research, although other private blockchain platforms can also be used. The consortium is established among edge servers from different domains, with each domain having a copy of the ledger in its edge server. The implemented blockchain design

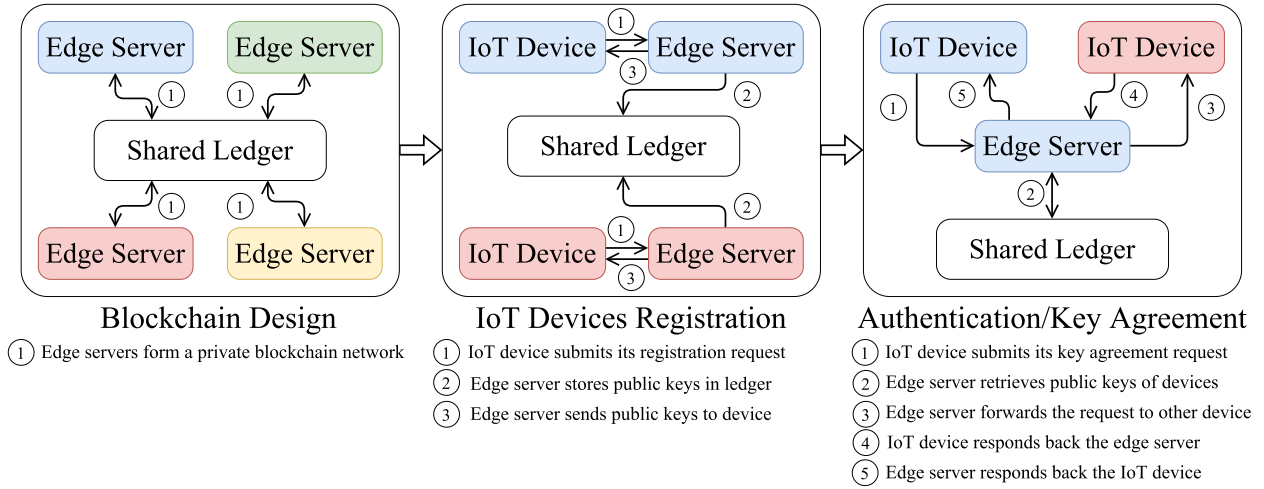


Fig. 2. Overview of the proposed protocol.

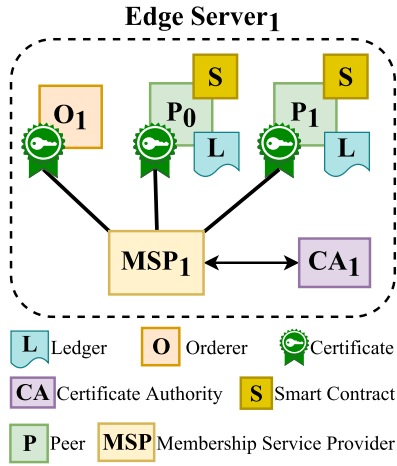


Fig. 3. Configuration of an edge server in our blockchain network design.

includes seven IoT domains with two peers and an orderer in each domain, as shown in Fig. 3. The peers host ledger and smart contract instances, and the blockchain channel allows communication between a specific group of peers and applications. The membership service provider (MSP) component manages the identities, certificates, and roles of participating peers and orderers in a domain, as well as their access level to the ledger. Edge servers send queries to endorsing peers, receive responses to their proposals, and send endorsed proposals to the corresponding orderer. The orderer packages the endorsed transactions into new blocks and broadcasts them to all the participating peers in the blockchain channel. Each peer validates the received transactions before committing them to its ledger. The entire transaction flow uses the Raft consensus mechanism [33].

In the proposed blockchain design, each IoT device's record is stored on the distributed ledger, containing the device's identification and public keys. This ensures that edge servers can verify the authenticity of the public key they read and that it

Algorithm 1: Smart Contract of the Designed Protocol.

Function WRITE ($ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2$):

```

    if  $ID_{d_i}$  then
        return  $ID_{d_i}$  already exists!
    else
        Insert ( $ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2, False$ ) into ledger
    end

```

Function READ (ID_{d_i}):

```

    if  $!ID_{d_i}$  then
        return  $ID_{d_i}$  does not exist!
    else
        if  $RevokeState == False$  then
            return ( $PK_{d_i}^1, PK_{d_i}^2$ ).
        end
    end
end

```

Function

UPDATE ($ID_{d_i}, newPK_{d_i}^1, newPK_{d_i}^2, newRevokeState$):

```

    if  $!ID_{d_i}$  then
        return  $ID_{d_i}$  does not exist!
    else
         $PK_{d_i}^1 = newPK_{d_i}^1$ 
         $PK_{d_i}^2 = newPK_{d_i}^2$ 
         $RevokeState = newRevokeState$ 
    end
end

```

Function REVOKE (ID_{d_i}):

```

    if  $!ID_{d_i}$  then
        return  $ID_{d_i}$  does not exist!
    else
        UPDATE( $ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2, True$ )
    end
end

```

has not been revoked. Algorithm 1 details the smart contract designed to perform write, read, update, and revoke functions on a record. Using the blockchain's distributed ledger to store public keys in the proposed protocol makes modifying or revoking them easy. The update or revoke function of the smart contract can be called to change or revoke the public keys of an IoT device.

TABLE III
DEVICE REGISTRATION

Device _i	Edge Server _i
Selects $sk_{d_i} \in Z_q^n$, where $\ sk_{d_i}\ \leq \beta$ Computes $PK_{d_i}^1 = sk_{d_i}^T \cdot M$ Computes $PK_{d_i}^2 = M \cdot sk_{d_i}$ $\{ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2\}$	Checks ID_{d_i} Inserts $(ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2, False)$ into the ledger Selects $sk_{S_i} \in Z_q^n$, where $\ sk_{S_i}\ \leq \beta$ Computes $PK_{S_i}^1 = sk_{S_i}^T \cdot M$ Computes $PK_{S_i}^2 = M \cdot sk_{S_i}$ $\{PK_{S_i}^1, PK_{S_i}^2\}$
Stores $(PK_{d_i}^1, PK_{d_i}^2, PK_{S_i}^1, PK_{S_i}^2)$	

TABLE IV
AUTHENTICATION AND KEY AGREEMENT

Device ₁ ($ID_{d_1}, PK_{d_1}^1, PK_{d_1}^2, sk_{d_1}$)	EdgeServer ₁ ($ID_{S_1}, PK_{S_1}^1, PK_{S_1}^2, sk_{S_1}$)	Device ₂ ($ID_{d_2}, PK_{d_2}^1, PK_{d_2}^2, sk_{d_2}$)
Chooses random $r_{d_1} \in Z_q^n$, where $\ r_{d_1}\ \leq \beta$ Computes $R_{d_1} = M \cdot r_{d_1}$ Computes $K_{d_1} = h_1(PK_{S_1}^1 \cdot r_{d_1})$ Computes $E_{d_1} = Enc_{(K_{d_1})}(ID_{d_1}, ID_{d_2})$ Computes $S_{d_1} = PK_{S_1}^1 \cdot sk_{d_1}$ Computes $H_{d_1} = h_2(ID_{d_1}, ID_{d_2}, R_{d_1}, K_{d_1}, E_{d_1}, S_{d_1}, T_{d_1})$ $\{R_{d_1}, E_{d_1}, H_{d_1}, T_{d_1}\}$	Checks $ T_{d_1} - T_{S_1} \leq \Delta t$ Computes $K_{d_1} = h_1(sk_{S_1}^T \cdot R_{d_1})$ Computes $(ID_{d_1}, ID_{d_2}) = Dec_{(K_{d_1})}(E_{d_1})$ Retrieves $PK_{d_1}^2$ and $PK_{d_1}^1$ Computes $S_{d_1} = sk_{S_1}^T \cdot PK_{d_1}^2$ Checks $h_2(ID_{d_1}, ID_{d_2}, R_{d_1}, K_{d_1}, E_{d_1}, S_{d_1}, T_{d_1}) \stackrel{?}{=} H_{d_1}$ Computes $K_{S_1} = h_1(PK_{d_2}^1 \cdot sk_{S_1})$ Computes $E_{S_1} = Enc_{(K_{S_1})}(ID_{d_1})$ Computes $H_{S_1} = h_2(ID_{S_1}, ID_{d_1}, ID_{d_2}, R_{d_1}, K_{S_1}, E_{S_1}, T_{S_1})$ $\{ID_{S_1}, R_{d_1}, E_{S_1}, H_{S_1}, T_{S_1}\}$	Checks $ T_{S_1} - T_{d_2} \leq \Delta t$ Computes $K_{S_1} = h_1(sk_{d_2}^T \cdot PK_{S_1}^2)$ Computes $(ID_{d_1}) = Dec_{(K_{S_1})}(E_{S_1})$ Checks $h_2(ID_{S_1}, ID_{d_1}, ID_{d_2}, R_{d_1}, K_{S_1}, E_{S_1}, T_{S_1}) \stackrel{?}{=} H_{S_1}$ Chooses random $r_{d_2} \in Z_q^n$, where $\ r_{d_2}\ \leq \beta$ Computes $R_{d_2} = r_{d_2}^T \cdot M$ $DH_{d_2} = r_{d_2}^T \cdot R_{d_1}$ $SSK_2 = h_2(ID_{d_1}, ID_{d_2}, DH_{d_2})$ $H_{d_2}^1 = h_2(ID_{S_1}, ID_{d_1}, ID_{d_2}, R_{d_2}, K_{S_1})$ $H_{d_2}^2 = h_2(ID_{d_1}, ID_{d_2}, R_{d_2}, DH_{d_2}, SSK_2)$ $\{R_{d_2}, H_{d_2}^1, H_{d_2}^2\}$
Computes $DH_{d_1} = R_{d_2} \cdot r_{d_1}$ Computes $SSK_1 = h_2(ID_{d_1}, ID_{d_2}, DH_{d_1})$ Checks $h_2(ID_{d_1}, ID_{d_2}, R_{d_2}, DH_{d_2}, SSK_1) \stackrel{?}{=} H_{d_2}^2$	Checks $h_2(ID_{S_1}, ID_{d_1}, ID_{d_2}, R_{d_2}, K_{S_1}) \stackrel{?}{=} H_{d_2}^1$ $\{R_{d_2}, H_{d_2}^1, H_{d_2}^2\}$	

B. IoT Device Registration

During this phase, an IoT device that wishes to communicate with other devices undergoes a series of operations, all of which are carried out by the upper edge servers of the IoT devices through secure channels. This process ensures that the IoT device is registered securely and can participate in subsequent communication processes with other registered devices. Table III illustrates the process of registering IoT devices. To begin, the i th IoT device, $Device_i$, in the domain of the i th edge server, $Edge Server_i$, selects its private key $sk_{d_i} \in Z_q^n$, where $\|sk_{d_i}\| \leq \beta$, and calculates its public keys $PK_{d_i}^1 = sk_{d_i}^T \cdot M$ and $PK_{d_i}^2 = M \cdot sk_{d_i}$, where $M \in Z_q^{n \times n}$ is the public matrix. Next, $Device_i$ submits its ID and public keys to $Edge Server_i$. After that, $Edge Server_i$ verifies the validity of ID_{d_i} and, if it is valid, uses the write function of the smart contract (shown in Algorithm 1) to add the $(ID_{d_i}, PK_{d_i}^1, PK_{d_i}^2, False)$ record to the ledger. The last field of the record determines whether the IoT device is revoked or not. Other edge servers will refer to this record in the subsequent phase. Finally, $Edge Server_i$ computes its public keys and transmits them back to $Device_i$.

C. Authentication/Key Agreement

During this stage, there is a mutual authentication process between $Device_1$ and $Device_2$, which results in an agreement on a session key $SSK_1 = SSK_2$. The details of this phase are presented in Table IV and elaborated further below.

$Device_1$ starts by selecting a random vector $r_{d_1} \in Z_q^n$, where $\|r_{d_1}\| \leq \beta$. After that, it computes R_{d_1} by performing a matrix-to-vector product between M and r_{d_1} . Afterward, an ephemeral secret key represented as K_{d_1} is computed to encrypt the IDs of both $Device_1$ and $Device_2$. The computation of K_{d_1} involves the public key of $Edge Server_1$, denoted as $PK_{S_1}^1$. It is important to mention that only $Edge Server_1$ has the ability to reconstruct K_{d_1} in the future, as this process requires the utilization of $Edge Server_1$'s private key. $Device_1$ employs its private key to compute S_{d_1} , which verifies its authenticity to the $Edge Server_1$. Next, H_{d_1} is calculated for the purpose of ensuring integrity. Finally, $Device_1$ sends $\{R_{d_1}, E_{d_1}, H_{d_1}, T_{d_1}\}$ to $Edge Server_1$.

After receiving the message from $Device_1$, the edge server validates the time stamp. If the time stamp is deemed valid, the server uses its secret key sk_{S_1} to compute the

ephemeral secret key K_{d_1} and then obtains the IDs of both the IoT devices. Next, the server retrieves the public keys $PK_{d_1}^1, PK_{d_1}^2, PK_{d_2}^1$, and $PK_{d_2}^2$ of the IoT devices from the blockchain. Using the public key $PK_{d_1}^2$, the server computes S_{d_1} to authenticate $Device_1$ and verifies the integrity token H_{d_1} . If the integrity token is valid, the server calculates the ephemeral secret key K_{S_1} to encrypt ID_{d_1} for $Device_2$. Finally, the server computes H_{S_1} and sends $\{ID_{S_1}, R_{d_1}, E_{S_1}, H_{S_1}, T_{S_1}\}$ to $Device_2$.

Upon receiving the message from the edge server, $Device_2$ validates the time stamp. If the time stamp is valid, the device employs its secret key, sk_{d_1} , to compute the ephemeral secret key, K_{S_1} , and retrieve ID_{d_1} . Following the integrity token check of H_{S_1} , $Device_2$ selects a random vector $r_{d_2} \in Z_q^n$ in a similar manner to $Device_1$. Next, $Device_2$ computes R_{d_2} by performing a vector-to-matrix product between $r_{d_2}^T$ and M . Then, it computes a Diffie–Hellman value, DH_{d_2} , and utilizes this value to compute the session key, SSK_2 . Finally, it computes two hash values ($H_{d_2}^1$ and $H_{d_2}^2$) and sends them, along with R_{d_2} , back to the edge server.

The authenticity and integrity of the received message is ensured by the edge server through checking $H_{d_2}^1$. If it is found to be valid, $\{R_{d_2}, H_{d_2}^2\}$ is sent to $Device_1$.

After receiving the message from the edge server, $Device_1$ calculates the Diffie–Hellman value DH_{d_1} based on the received R_{d_2} . It then uses this value to compute the session key SSK_1 . Finally, $Device_1$ verifies $H_{d_2}^2$ to ensure that it has the same session key as $Device_2$.

V. CORRECTNESS AND SECURITY ANALYSES

This section presents the correctness of the proposed protocol along with both the informal and formal security proofs.

A. Correctness

Theorem 1: Secret key K_{d_1} is shared between $Device_1$ and $Edge Server_1$ during the authentication. The generated K_{d_1} by $Device_1$ is identical to the generated one by $Edge Server_1$.

Proof:

$$\begin{aligned} K_{d_1}(Device_1) &= h_1(PK_{S_1}^1 \cdot r_{d_1}) \\ &= h_1((PK_{S_1}^1) \cdot r_{d_1}) = h_1((sk_{S_1}^T \cdot M) \cdot r_{d_1}) \\ &= h_1(sk_{S_1}^T \cdot (M \cdot r_{d_1})) = h_1(sk_{S_1}^T \cdot (R_{d_1})) \\ &= h_1(sk_{S_1}^T \cdot R_{d_1}) = K_{d_1}(Edge Server_1). \end{aligned} \quad (1)$$

Theorem 2: Secret key K_{S_1} is shared between $Device_2$ and $Edge Server_1$ during the authentication. The generated K_{S_1} by $Device_2$ is identical to the generated one by $Edge Server_1$.

Proof:

$$\begin{aligned} K_{S_1}(Edge Server_1) &= h_1(PK_{d_2}^1 \cdot sk_{S_1}) \\ &= h_1((PK_{d_2}^1) \cdot sk_{S_1}) = h_1((sk_{d_2}^T \cdot M) \cdot sk_{S_1}) \\ &= h_1(sk_{d_2}^T \cdot (M \cdot sk_{S_1})) = h_1(sk_{d_2}^T \cdot (PK_{S_1}^2)) \end{aligned}$$

$$= h_1(sk_{d_2}^T \cdot PK_{S_1}^2) = K_{S_1}(Device_2). \quad (2)$$

Theorem 3: Both the IoT devices share the same session key, i.e., $SSK_1 = h_2(ID_{d_1}, ID_{d_2}, DH_{d_1})$ is same as $SSK_2 = h_2(ID_{d_1}, ID_{d_2}, DH_{d_2})$.

Proof: As proved in (3), because Diffie–Hellman values of both the IoT devices are identical, i.e., $DH_{d_2} = DH_{d_1}$, we conclude that $SSK_2 = SSK_1$.

$$\begin{aligned} DH_{d_2} &= r_{d_2}^T \cdot R_{d_1} = r_{d_2}^T \cdot (R_{d_1}) = r_{d_2}^T \cdot (M \cdot r_{d_1}) \\ &= (r_{d_2}^T \cdot M) \cdot r_{d_1} = (R_{d_2}) \cdot r_{d_1} = R_{d_2} \cdot r_{d_1} = DH_{d_1}. \end{aligned} \quad (3)$$

B. Informal Justification

This subsection demonstrates the protocol's ability to withstand known security attacks and its effectiveness in providing critical security features by offering informal explanations.

1) *Anonymity and Unlinkability:* Unlike previous quantum-resistant protocols that lack strong anonymity, the proposed protocol offers both anonymity and unlinkability. Rather than transmitting the actual identity (ID_{d_i}) of devices to servers, the encrypted form of ID_{d_i} , denoted as E_{d_i} , is sent, ensuring anonymity. In addition, the encryption key K_{d_i} is randomly altered for each session, rendering the protocol unlinkable.

2) *Perfect Forward Secrecy:* The session key is derived from the temporary session parameters r_{d_1} and r_{d_2} . This ensures that even if the secret keys are compromised, the temporary session parameters remain unknown to any potential adversary. As a result, the protocol supports perfect forward secrecy, since previously intercepted sessions cannot be compromised even if the secret keys are later revealed.

3) *No Key Escrow:* Our protocol avoids this issue by ensuring that the secret information of IoT devices is only known to the devices themselves, without any access given to upper trusted edge servers.

4) *Impersonation and Modification:* An adversary attempting to impersonate or modify the communication would be unsuccessful because they lack the ability to compute $S_{d_1} = PK_{S_1}^1 \cdot sk_{d_1}$ or $K_{S_1} = h_1(sk_{d_2}^T \cdot PK_{S_1}^2)$, which is necessary to generate a valid hashed token. In both the scenarios, a secret key from one of the IoT devices is required.

5) *Replay:* The time stamp (T_{d_1}) is initially verified when it is sent to the server. In addition, T_{d_1} is utilized in H_{d_1} and forwarded to the server to prevent the occurrence of replayed messages. It is important to note that it is not possible for the adversary to modify the time. The same process occurs when $Edge Server_1$ sends a message to $Device_2$.

6) *Ephemeral Secret Leakage:* Even when the temporary parameters of a session (r_{d_1} and r_{d_2}) are disclosed, the adversary remains incapable of computing the session key without knowledge of the IDs of the IoT devices. Acquiring access to the secret key of one of the IoT devices (sk_{d_1} or sk_{d_2}) is necessary to obtain the IDs of the IoT devices.

C. Random Oracle Proof

In this section, we provide a formal proof of the proposed protocol using the random oracle model. The aim is to demonstrate the resistance of the proposed quantum-proof authentication and key agreement protocol, denoted as $QAKA$, against well-known attacks. To this end, we define a game, denoted as $G_{QAKA}(\mathcal{A}, Device_O, Edge_Server_O)$, in which a time-limited CK attacker $\mathcal{A}$ interacts with oracles $Device_O$ and $Edge_Server_O$, where $Device_O$ can be an instance of $Device_1$ or $Device_2$. During the game, $\mathcal{A}$ can issue several queries as defined in Table II. However, it is important to note that the $TST()$ query can only be issued once. Let $Succ_{\mathcal{A}}^{QAKA}$ represent the success of $\mathcal{A}$ in the game. The advantage of $\mathcal{A}$ in breaking the protocol is denoted as $Adv_G^{Break} = |2Pr(Succ_{\mathcal{A}}^{QAKA}) - 1|$. If $\mathcal{A}$'s breaking advantage is a negligible amount, the protocol can be considered secure. We prove that $Adv_G^{Break} < \varepsilon$ through the following theorem.

Theorem 4: Given that the number of queries for the $SND()$, $HSH()$, and $EXE()$ operations is represented by s , h , and e , respectively, the value of Adv_G^{Break} is expressed as follows:

$$Adv_G^{Break} \leq 2(Adv_{\mathcal{A}}^{AES} + Adv_{\mathcal{A}}^{Cbi-ISIS}) + \frac{(s+e)^2}{q} + \frac{2(s+2h_2^2)}{|H_2|} + \frac{2h_1^2}{|H_1|} \quad (4)$$

where $Adv_{\mathcal{A}}^{AES}$ and $Adv_{\mathcal{A}}^{Cbi-ISIS}$ refer to the advantages that $\mathcal{A}$ has in breaking AES-256 and Cbi-ISIS [8], respectively.

Proof: To prove the theorem mentioned earlier, we can specify subgames from G_0 to G_5 . The success of $\mathcal{A}$ in G_n can be denoted by $Succ_{\mathcal{A}}^{G_n}$. An event is considered a success event if $\mathcal{A}$ can correctly guess a particular bit when the $TST()$ query is made. The definitions for each subgame are as follows.

G_0 : As this subgame is indicative of a real attack scenario, we can deduce the following equation:

$$Adv_G^{Break} = |2Pr(Succ_{\mathcal{A}}^{G_0}) - 1|. \quad (5)$$

G_1 : In this subgame, the oracle instances respond to $\mathcal{A}$ using the suggested protocol description, leading to the deduction of the following equation:

$$Pr(Succ_{\mathcal{A}}^{G_0}) = Pr(Succ_{\mathcal{A}}^{G_1}). \quad (6)$$

G_2 : This subgame differs from the previous one in the event of a transcript or hash collision. As the probability of a hash collision is $h^2/|H|$, and the transcript collision probability is $(s+e)^2/2q$, the following equation can be derived:

$$|Pr(Succ_{\mathcal{A}}^{G_2}) - Pr(Succ_{\mathcal{A}}^{G_1})| \leq h_1^2/|H_1| + h_2^2/|H_2| + (s+e)^2/2q. \quad (7)$$

G_3 : This subgame differs from the previous one in that it involves the correct guessing of the values of h_2 by $\mathcal{A}$, without requiring an $HSH()$ query to be sent. As a result of this guessing, the following equation can be derived:

$$|Pr(Succ_{\mathcal{A}}^{G_3}) - Pr(Succ_{\mathcal{A}}^{G_2})| \leq s/|H_2|. \quad (8)$$

G_4 : In this subgame, we demonstrate that our proposed system is resistant to the leakage of both the long- and short-term secrets.

In the case of long-term secrets, $\mathcal{A}$ attempts to obtain them by making $LTSR()$ queries, while for short-term secrets, the queries made are $STSR()$ in nature. However, even in these scenarios, $\mathcal{A}$ is unable to obtain the session key without breaking either AES-256 or Cbi-ISIS, which leads to the derivation of the following equation:

$$|Pr(Succ_{\mathcal{A}}^{G_4}) - Pr(Succ_{\mathcal{A}}^{G_3})| \leq Adv_{\mathcal{A}}^{AES} + Adv_{\mathcal{A}}^{Cbi-ISIS}. \quad (9)$$

G_5 : This subgame is terminated when $\mathcal{A}$ sends an $ID()$ query to obtain the identifiers of the entities or makes an $HSH()$ query to acquire the session key. This leads to the derivation of the following equation:

$$|Pr(Succ_{\mathcal{A}}^{G_5}) - Pr(Succ_{\mathcal{A}}^{G_4})| \leq Adv_{\mathcal{A}}^{AES} + h_2^2/|H_2|. \quad (10)$$

The proof of Theorem 1 is accomplished by combining (5)–(10) in the following manner:

$$\begin{aligned} Adv_G^{Break} &= 2Pr(Succ_{\mathcal{A}}^{G_0}) - 1 = 2Pr(Succ_{\mathcal{A}}^{G_5}) - 1 \\ &+ 2(Pr(Succ_{\mathcal{A}}^{G_0}) - Pr(Succ_{\mathcal{A}}^{G_5})) \leq 2Pr(Succ_{\mathcal{A}}^{G_5}) - 1 \\ &+ 2 \sum_{n=0}^4 (Pr(Succ_{\mathcal{A}}^{G_{n+1}}) - Pr(Succ_{\mathcal{A}}^{G_n})) = 2 \times \frac{1}{2} - 1 \\ &+ 2(Adv_{\mathcal{A}}^{AES} + h_2^2/|H_2| + Adv_{\mathcal{A}}^{AES} + Adv_{\mathcal{A}}^{Cbi-ISIS} + s/|H_2| \\ &+ h_1^2/|H_1| + h_2^2/|H_2| + (s+e)^2/2q) = 2(Adv_{\mathcal{A}}^{AES} \\ &+ Adv_{\mathcal{A}}^{Cbi-ISIS}) + \frac{(s+e)^2}{q} + \frac{2(s+2h_2^2)}{|H_2|} + \frac{2h_1^2}{|H_1|}. \end{aligned} \quad (11)$$

VI. PERFORMANCE EVALUATION

This section provides a detailed description of the research implementation and presents the results of our comparative analysis.

A. Implementation

For this investigation, we employed the ESP32-DevKitC as the IoT device. The ESP32-DevKitC is a popular development board for IoT projects, with dual-core processors, WiFi and Bluetooth connectivity, and various peripherals. It is easy to use, supports battery operation, and can be programmed using Arduino IDE, MicroPython, or ESP-IDF. To conduct cryptographic tests on the ESP32-DevKitC board, we employed the “Arduino” cryptography library [35]. The results of these tests are presented in Table V, which provides the execution times of various cryptographic operations. The matrix dimension is presented as an $n \times n$ matrix, while the vector is represented as a $1 \times n$ or $n \times 1$ vector, with n set to 64. Furthermore, the matrix and vector entries are random numbers within the range of 0 to $q-1$, where q is fixed at 4096 [17].

B. Blockchain Implementation

The applied blockchain architecture, as shown in Fig. 3, was implemented on an HP Proliant DL380 G9 Server with 10 CPU cores and 16 GB of RAM, running on a virtual Ubuntu machine. The smart contract, described in Algorithm 1, was

TABLE V
 EXECUTION TIME OF CRYPTOGRAPHIC OPERATIONS

Operation	Symbol	Exe. IoT	Exe. Edge
Point multiplication in ECC	T_p	25,020 μ s	77 μ s
Chebyshev polynomial	T_{ch}	8,562 μ s	26.35 μ s
Point addition in ECC	T_a	103 μ s	0.41 μ s
Hash (SHA-256)	T_h	30 μ s	0.31 μ s
Symmetric encryption (AES-256)	T_{se}	10 μ s	0.14 μ s
Matrix to matrix multiplication	T_{mm}	192,175 μ s	549.1 μ s
Vector to matrix multiplication	T_{vm}	3,017 μ s	10.16 μ s
Vector to vector multiplication	T_{vv}	45 μ s	0.16 μ s
Vector addition	T_{va}	31 μ s	0.1 μ s
Scalar vector multiplication	T_{sv}	30 μ s	0.1 μ s
Exponentiation	T_e	59,850 μ s	212.9 μ s
Bilinear pairing	T_{bp}	275,726 μ s	925.26 μ s
Reading from blockchain	T_{BC}^R	—	1.85 ms
Writing on blockchain	T_{BC}^W	—	12.32 ms

Edge Server: HP Proliant DL380 G9 Server with ten CPU cores and 16 GB of RAM, IoT Device: ESP32-DevKitC.

 TABLE VI
 HYPERLEDGER CALIPER RESULTS

Name	Success	Fail	Throughput (TPS)
Write	850	0	81.2
Read	5482	0	541.6
Update	308	0	30.7

created using the GO programming language and installed on all the blockchain peers. To evaluate the performance of the distributed ledger, the Caliper benchmarking tool was used to measure the time required for read, update, and insert operations. The Raft consensus mechanism was employed in this study to ensure that all the network participants have the same view of the distributed ledger's state at any given time. The results in Table VI show that read operations were completed in an average time of 1.85 ms, whereas update and write operations took longer, with an average of 32.57 and 12.32 ms, respectively. These findings suggest that updating and inserting data into the blockchain may require more computational resources and time than reading from it. However, given the infrequency of update and insert operations, their performance is less critical to the overall efficiency of the system. It is important to note that these performance results are subject to variation based on factors, such as network size, number of nodes, smart contract implementation, and the use of different consensus mechanisms.

C. Comparative Efficiency Analysis

A thorough comparative analysis has been conducted to prove that the proposed approach is better than the current solutions. The analysis involved evaluating both the classic and post-quantum solutions for comparison. While classical techniques have been around for a while and can handle complex security features, post-quantum approaches are new and require substantial development. Another reason for this analysis was to gain a clear understanding of the benefits and drawbacks of using post-quantum primitives.

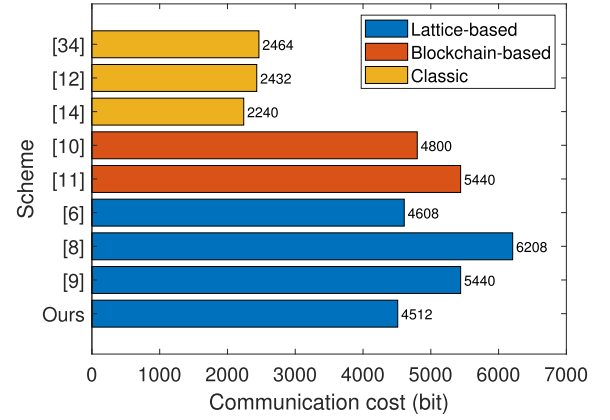


Fig. 4. Communication cost comparison.

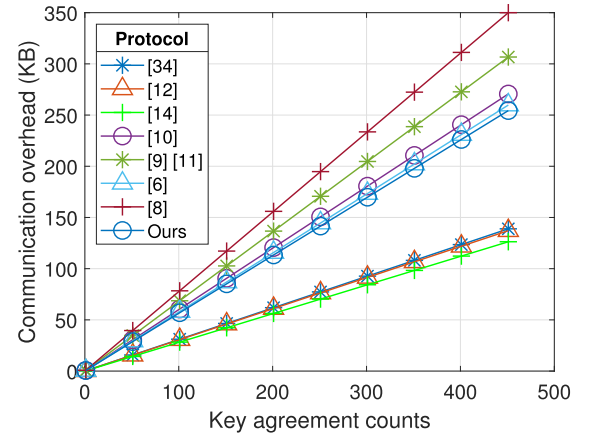


Fig. 5. Communication costs of related works for different key agreement counts.

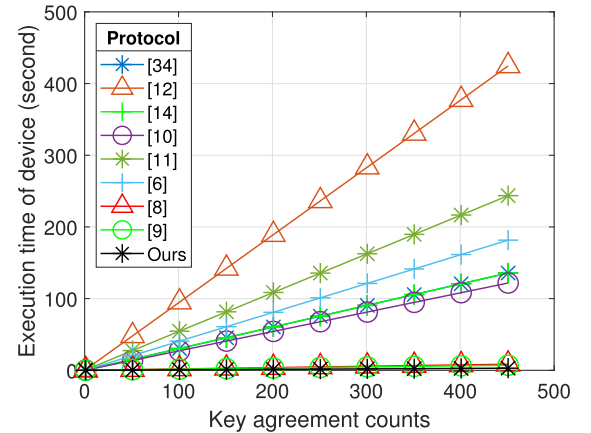


Fig. 6. Computation costs of related works for different key agreement counts.

Table VII compares the proposed authentication protocol with classical and post-quantum methods in terms of computation overhead on IoT devices and edge servers. One of the key factors contributing to the superiority of the proposed protocol over its counterparts in terms of IoT device computation cost

TABLE VII
COMPUTATION COST COMPARISON

Scheme	Device (Cost in μs)	Server (Cost in μs)
Classic approaches		
[34]	$12T_p + 4T_a + 10T_h$ (300,952)	-
[9]	$2T_{bp} + 4T_e + 6T_p + 2T_a + 10T_h$ (941,478)	-
[11]	$12T_p + 4T_a + 10T_h$ (300,952)	-
Blockchain-based approaches		
[19]	$2T_e + 6T_p + 6T_h$ (270,000)	$4T_{bp} + 6T_e + 12T_p + 4T_a + 14T_h + 2T_{BC}^R + 2T_{BC}^W$ (5,936.76)
[5]	$4T_e + 12T_p + 18T_h$ (540,180)	$4T_{bp} + 2T_e + 18T_p + 8T_a + 20T_h + 2T_{BC}^R$ (5,526.02)
Lattice-based approaches		
[6]	$2T_{mm} + 6T_{vm} + 2T_{vv} + 2T_{se} + 2T_{sd} + 2T_{va}$ (402,644)	-
[18]	$6T_{vm} + 4T_{vv} + 6T_{sv} + 4T_h + 6T_{va}$ (18,768)	-
[8]	$5T_{vm} + 2T_{vv} + 6T_{sv} + 4T_h + 4T_{va}$ (15,599)	-
Ours	$2T_{vm} + 5T_{vv} + 9T_h + T_{se} + T_{sd}$ (6,549)	$3T_{vv} + T_{se} + T_{sd} + 5T_h + 2T_{BC}^R$ (6.01)

All the parameters are specified in Table V.

TABLE VIII
STORAGE OVERHEAD COMPARISON

Scheme	Storage on the IoT device	Cost (bit)
[6]	$(2 + N_d)n^2q_b$	$49,152N_d + 98,304$
[18]	$(n^2 + N_d(n+1))q_b$	$780N_d + 49,152$
[8]	$(n^2 + N_dn)q_b$	$768N_d + 49,152$
Ours	$2nq_b$	1,536

$q_b = 12$, $n = 64$, N_d : Number of IoT devices.

is the shifting of device authentication and reading from the blockchain burden to the edge servers.

Since lattice-based approaches require significantly more storage than nonlattice-based approaches, we only compare lattice-based approaches, including our proposed solution. Table VIII presents the comparison results, with a specific focus on the overhead for IoT devices. The proposed protocol has a significantly lower storage overhead compared to other approaches. This is because, unlike other lattice-based approaches, IoT devices in our protocol only need to store the public key of the edge server in their domain. In contrast, other approaches require the storage of the public key of all the IoT devices, which incurs a high cost in lattice cryptography.

The communication cost for various protocols is displayed in Fig. 4. The proposed protocol demonstrates superior performance compared to other lattice-based approaches. However, it is worth noting that lattice-based approaches typically require larger private and public keys than classical approaches, resulting in higher communication costs. For our experiments, we utilized the same parameter sizes for all the approaches, including 32-bit ID and time stamp, 256-bit hash, 160-bit ECC signature, 320-bit ECC public key, n as 64, and q as 12 bits. We have chosen these parameter sizes because they are the most common sizes, which have been used in the literature [17], [36]. Figs. 5 and 6, respectively, illustrate the communication and computation costs of related works for different key agreement counts.

VII. CONCLUSION

In this article, we introduced a novel distributed authentication and key exchange protocol based on lattice cryptography and blockchain for securing D2D communications. The protocol addressed many security and privacy challenges, including high storage costs, a lack of strong anonymity, and high key revocation overhead. Its proven resilience against several known classical and post-quantum security attacks, as well as its resource efficiency and lower overhead, made the protocol an ideal solution for securing D2D communication in resource-constrained IoT environments. Furthermore, the protocol offers various benefits, such as interdomain authentication support, and it will enhance the security and efficiency of D2D communications in IoT, with potential applications in domains, such as smart homes, industrial automation, health care, and vehicle-to-vehicle communications. For future work, we plan to improve the communication overhead of the protocol. We also intend to extend this work to peer-to-peer networks and fifth/sixth-generation wireless networks.

REFERENCES

- [1] S. Azizi, M. Shojafar, J. Abawajy, and R. Buyya, "Deadline-aware and energy-efficient IoT task scheduling in fog computing systems: A semi-greedy approach," *J. Netw. Comput. Appl.*, vol. 201, 2022, Art. no. 103333.
- [2] H. Zhang et al., "Secure edge-aided computations for social Internet-of-Things systems," *IEEE Trans. Comput. Social Syst.*, vol. 9, no. 1, pp. 76–87, Feb. 2022.
- [3] F. Li et al., "A measurement study on device-to-device communication technologies for IIoT," *Comput. Netw.*, vol. 192, 2021, Art. no. 108072.
- [4] A. Braeken, "Device-to-device group authentication compatible with 5G AKA protocol," *Comput. Netw.*, vol. 201, 2021, Art. no. 108575.
- [5] Y. Zhang, B. Li, J. Wu, B. Liu, R. Chen, and J. Chang, "Efficient and privacy-preserving blockchain-based multi-factor device authentication protocol for cross-domain IIoT," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22501–22515, Nov. 2022.
- [6] D. S. Gupta and G. Biswas, "A novel and efficient lattice-based authenticated key exchange protocol in C-K model," *Int. J. Commun. Syst.*, vol. 31, no. 3, 2018, Art. no. e3473.
- [7] C. Pu, A. Wall, K.-K. R. Choo, I. Ahmed, and S. Lim, "A lightweight and privacy-preserving mutual authentication and key agreement protocol for internet of drones environment," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9918–9933, Jun. 2022.

- [8] D. S. Gupta, S. Ray, T. Singh, and M. Kumari, "Post-quantum lightweight identity-based two-party authenticated key exchange protocol for Internet of Vehicles with probable security," *Comput. Commun.*, vol. 181, pp. 69–79, 2022.
- [9] B. Semal, K. Markantonakis, and R. N. Akram, "A certificateless group authenticated key agreement protocol for secure communication in untrusted UAV networks," in *Proc. IEEE/AIAA 37th Digit. Avionics Syst. Conf.*, 2018, pp. 1–8.
- [10] B. Bera, D. Chattaraj, and A. K. Das, "Designing secure blockchain-based access control scheme in IoT-enabled internet of drones deployment," *Comput. Commun.*, vol. 153, pp. 229–249, 2020.
- [11] A. Irshad, S. A. Chaudhry, A. Ghani, and M. Bilal, "A secure blockchain-oriented data delivery and collection scheme for 5G-enabled IoD environment," *Comput. Netw.*, vol. 195, 2021, Art. no. 108219.
- [12] S. A. Eftekhari, M. Nikooghadam, and M. Rafighi, "Security-enhanced three-party pairwise secret key agreement protocol for fog-based vehicular ad-hoc communications," *Veh. Commun.*, vol. 28, 2021, Art. no. 100306.
- [13] B. Seok, J. C. S. Sicato, T. Erzhen, C. Xuan, Y. Pan, and J. H. Park, "Secure D2D communication for 5G IoT network based on lightweight cryptography," *Appl. Sci.*, vol. 10, no. 1, 2019, Art. no. 217.
- [14] D. Dey, S. Chandra, and N. Ghosh, "HessianAuth: An ECC-based distributed and efficient authentication mechanism for 6LoWPAN networked IoT devices," in *Proc. 24th Int. Conf. Distrib. Comput. Netw.*, 2023, pp. 227–236.
- [15] S. Kumari, M. Singh, R. Singh, and H. Tewari, "A post-quantum lattice based lightweight authentication and code-based hybrid encryption scheme for IoT devices," *Comput. Netw.*, vol. 217, 2022, Art. no. 109327.
- [16] D. S. Gupta, S. H. Islam, M. S. Obaidat, A. Karati, and B. Sadoun, "LAAC: Lightweight lattice-based authentication and access control protocol for E-health systems in IoT environments," *IEEE Syst. J.*, vol. 15, no. 3, pp. 3620–3627, Sep. 2021.
- [17] S. Wang, Y. Zhu, D. Ma, and R. Feng, "Lattice-based key exchange on small integer solution problem," *Sci. China Inf. Sci.*, vol. 57, no. 11, pp. 1–12, 2014.
- [18] S. H. Islam and S. Zeadally, "Provably secure identity-based two-party authenticated key agreement protocol based on CBI-ISIS and Bi-ISIS problems on lattices," *J. Inf. Secur. Appl.*, vol. 54, 2020, Art. no. 102540.
- [19] M. Shen et al., "Blockchain-assisted secure device authentication for cross-domain industrial IoT," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 5, pp. 942–954, May 2020.
- [20] G. Bansal and B. Sikdar, "S-MAPS: Scalable mutual authentication protocol for dynamic UAV swarms," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12088–12100, Nov. 2021.
- [21] V. S. Naresh, S. Reddi, S. Kumari, V. D. Allavarpu, S. Kumar, and M.-H. Yang, "Practical identity based online/off-line signcryption scheme for secure communication in Internet of Things," *IEEE Access*, vol. 9, pp. 21267–21278, 2021.
- [22] A. Elkhailil, J. Zhang, R. Elhabob, and N. Eltayieb, "An efficient signcryption of heterogeneous systems for Internet of Vehicles," *J. Syst. Archit.*, vol. 113, 2021, Art. no. 101885.
- [23] D. Dharminder, C. B. Reddy, A. K. Das, Y. Park, and S. S. Jamal, "Post-quantum lattice based secure reconciliation enabled key agreement protocol for IoT," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2680–2692, Feb. 2023.
- [24] J. Pan, J. Wang, A. Hester, I. Alqerm, Y. Liu, and Y. Zhao, "EdgeChain: An edge-IoT framework and prototype based on blockchain and smart contracts," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4719–4732, Jun. 2019.
- [25] Y. Jiang, X. Xu, H. Gao, A. D. Rajab, F. Xiao, and X. Wang, "LBlockchainE: A lightweight blockchain for edge IoT-enabled maritime transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2307–2321, Feb. 2023.
- [26] M. Zhang, J. Cao, Y. Sahni, Q. Chen, S. Jiang, and L. Yang, "Blockchain-based collaborative edge intelligence for trustworthy and real-time video surveillance," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1623–1633, Feb. 2023.
- [27] R. Canetti and H. Krawczyk, "Analysis of key-exchange protocols and their use for building secure channels," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2001, pp. 453–474.
- [28] D. S. Gupta and G. Biswas, "Design of lattice-based ElGamal encryption and signature schemes using SIS problem," *Trans. Emerg. Telecommun. Technol.*, vol. 29, no. 6, 2018, Art. no. e3255.
- [29] J. Ding, K. Schmitt, and Z. Zhang, "A key exchange based on the short integer solution problem and the learning with errors problem," in *Proc. Int. Conf. Codes, Cryptol., Inf. Secur.*, 2019, pp. 105–117.
- [30] A. Vangala, A. K. Sutrala, A. K. Das, and M. Jo, "Smart contract-based blockchain-envisioned authentication scheme for smart farming," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10792–10806, Jul. 2021.
- [31] M. J. Hossain, C. Xu, Y. Zhang, X. Zhang, and W. Li, "LAMA: A secure lattice-based authentication scheme for cloud storage against misbehaved private key generator," *J. Ambient Intell. Humanized Comput.*, vol. 14, pp. 8613–8629, 2023.
- [32] "Hyperledger fabric software," 2023. [Online]. Available: <https://www.hyperledger.org/use/fabric>
- [33] D. Huang, X. Ma, and S. Zhang, "Performance analysis of the raft consensus algorithm for private blockchains," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 1, pp. 172–181, Jan. 2020.
- [34] P. Bagga, A. K. Das, M. Wazid, J. J. Rodrigues, K.-K. R. Choo, and Y. Park, "On the design of mutual authentication and key agreement protocol in internet of vehicles-enabled intelligent transportation system," *IEEE Trans. Veh. Technol.*, vol. 70, no. 2, pp. 1736–1751, Feb. 2021.
- [35] "Arduino Cryptography Library," 2023. [Online]. Available: <https://github.com/rweather/arduinoilibs>
- [36] D. Abbasinezhad-Mood, S. M. Mazinani, M. Nikooghadam, and A. O. Sharif, "Efficient provably-secure dynamic ID-based authenticated key agreement scheme with enhanced security provision," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1227–1238, Mar./Apr. 2022.